

Parallel FPGA Routing: Survey and Challenges

Mirjana Stojilović

School of Computer and Communication Sciences

École Polytechnique Fédérale de Lausanne (EPFL), Switzerland

mirjana.stojilovic@epfl.ch

Abstract—As transistor scaling is slowing down [1], other opportunities for ensuring continuous performance increase have to be explored. Field programmable gate arrays (FPGAs) are in the spotlight these days: not only due to their malleability and energy efficiency, but also because FPGAs have recently been integrated into the *cloud* [2]. The latter makes them available to everyone in need of the immense computing power and data throughput they can offer. However, one important issue needs to be resolved first—the time to compile an industrial-scale design for an FPGA must be drastically reduced. Researchers have been looking for ways to accelerate FPGA routing through parallelism, since routing is one of the most time-consuming compilation steps. However, the ideal solution has not been found yet. This paper provides a survey of parallel FPGA routers, with the aim to identify their strengths and weaknesses, thus suggesting directions to take in further efforts for acceleration.

I. INTRODUCTION

FPGAs are a hot topic these days: as all the big datacenters are experiencing increasing demands to support heavy workloads, e.g., deep neural networks, complex data analytics, 4K video streaming, etc., the malleability and energy-efficiency of FPGAs have inspired big *cloud* providers to include FPGAs in their datacenters. Yet, two issues still stand in the way of massive FPGA adoption: (1) difficult programming and (2) long FPGA compilation time (from the high-level description to “*binary*”). While the first issue is the topic of extensive research and higher-level programming frameworks are and will continue to emerge, in this paper we focus on the second issue. These days, the time to compile an industrial-scale circuit lies in the order of hours or days [3], depending on its size. Knowing that a design is modified many times before it is ready for deployment, these long turnaround times not only seriously affect the time-to-market but further alienate FPGA designers from FPGAs. In the era of multicore processors, servers, and powerful *cloud* platforms, parallelization can dramatically cut the compilation time [4].

Routing, whether it is on a PCB, a VLSI circuit, or an FPGA, is a very time-consuming step. Researchers have recognized the importance of widening this bottleneck. There have been many attempts to develop parallel routers for standard-cell designs [5], but not as many for FPGAs. This paper surveys parallel FPGA routers, gives insights into advantages and drawbacks of each, compares them and proposes ideas on where to look for further acceleration opportunities.

A. FPGA Routing Problem

The primary data structure representing FPGA routing resources is the directed *Routing Resource Graph* (RRG)

$G = (V, E)$, where V is the set of vertices and E is the set of edges. Each vertex $v \in V$ represents a wire or a pin. Each edge $e_{ij} \in E$ represents a programmable connection point between a pin and a wire segment, or a programmable routing switch between two wire segments. Each signal i to route through G forms a net N_i , comprising one source vertex $s_i \in V$ and one or more sinks $t_{ij} \in V$. The solutions to the routing problem of the net N_i are paths connecting the source to all its sinks. These paths form a directed *routing tree* $RT(N_i) \subset G$. The RTs for different nets must be disjoint, to prevent short circuits.

Routing a design on a commercial-grade FPGA is a complex task, partly because an RRG can be extremely large, given the number of available programmable routing resources.

B. Negotiation-Based Router

In 1995, McMurchie and Ebeling presented PathFinder, an iterative algorithm that achieves a good compromise between two conflicting goals: eliminating congestion and minimizing critical path delay [6]. Its distinctive feature is the existence of a cost of using a given vertex v in a route. While signals compete for v , they *negotiate* and the cost evolves. This algorithm, also called *negotiation-based router*, is in the heart of the modern FPGA routers.

PathFinder implementation is a triple-nested loop. The outer loop, called the *global router*, invokes the middle loop, called the *signal router*, for all signals i to be routed. Each signal router iteration rips-up the existing routing tree and re-routes it by first inserting the sink s_i into $RT(N_i)$ and then invoking the inner loop, called *maze expansion* [7]. In each maze expansion iteration, i.e., for all sinks t_{ij} , the vertices in $RT(N_i)$ are inserted into a priority queue (PQ). Then, the lowest-cost vertex v_{min} in the PQ is selected. If v_{min} is a sink, a path is constructed by invoking a backtrace procedure and added to $RT(N_i)$. Otherwise, the search *expands* and all neighbors of v_{min} that have not been visited previously are inserted in the PQ. PathFinder terminates when there are no shared resources among the routing trees (no congestion), or after a fixed number of iterations fail. Since FPGA routing is NP-complete [8], PathFinder is not guaranteed to find a legal route, even if one exists. The algorithm is inherently sequential and, since it operates on graphs, it is irregular. These make it very challenging to parallelize.

The formula used for computing the cost of a vertex, when it is inserted in the PQ, is what distinguishes *routing-driven* from *timing-driven* routers. In routing-driven routers, the node cost depends on the intrinsic delay, past and present

congestion of the node [9]. In timing-driven routers, accurate delay calculation is crucial: detailed delay models, such as Elmore model [9], are used. In this model, the delay to a node depends on the capacitances and resistances present in the tree.

C. Parallel Computing Terminology

Before comparing parallel programming approaches, it is perhaps helpful to recall some of the very basics of parallel computing [10].

The main components of a parallel computer system are processors and memories. To allow for parallelization, if we exclude simultaneous multi-threading, there are always multiple CPUs, either on the same die (multi-core) or distributed. Memory can be *shared* or *distributed*. Shared memory (SM) allows each processor to access all memory as global address space. In distributed-memory (DM) systems, CPUs have local memories, connected through a communication network, and local address spaces. Even hybrid distributed-shared memory architectures exist, with the option to use GPUs.

The existing parallel FPGA routers employ the following parallel programming models:

- *SM with threads*. Threads can be seen as subroutines within a main program. Compared to the cost of creating and managing a process, threads are light-weight. One standard implementation of threads is POSIX threads. Besides access to the global SM, threads have their own private data. A common model for a threaded program is *manager/worker* in which a single thread assigns work to other threads. To protect the access to shared data, one can use mutexes (locks that can be shared by multiple processes), barriers (forcing threads to wait until all threads reach the barrier), spinlocks (continuously polling a mutex until it becomes free), or semaphores (allow multiple threads to obtain a lock simultaneously).
- *DM with message passing*, in which tasks run on multiple machines. The tasks have local memory. To exchange data, they send and receive messages. The industry standard for messaging is Message Passing Interface (MPI). It offers various types of send/receive routines, such as synchronous send (waiting on receive to return), blocking send (assuring that modifications cannot affect the data intended for the receive task), non-blocking send, etc.
- *Hybrid* models that are a combination of others, e.g., a combination of MPI with threads (suited for today's clustered many-core machines), or MPI with CPU-GPU.

II. PARALLEL FPGA ROUTING

Most of the work on FPGA routing focuses on accelerating PathFinder, despite its sequential nature. It is only recently that different approaches were proposed.

Almost all parallel routers are either *coarse-grain* or *fine-grain*. Coarse-grain routers distribute the problem by partitioning the nets; these partitions are then routed independently. We may say that they provide net-level parallelism. Their overall performance depends on how the partitioning is done

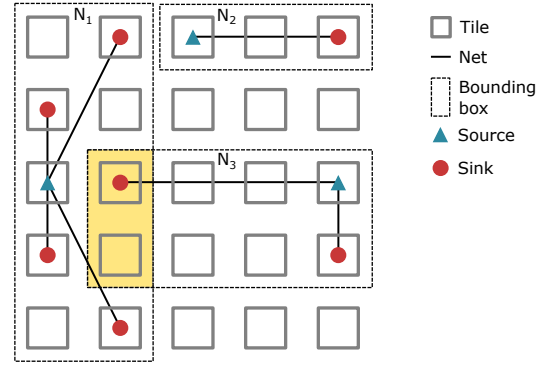


Fig. 1. FPGA tiles and three example nets. Net N_1 has higher fanout than nets N_2 and N_3 . Each net is surrounded by its rectangular bounding box, containing the net terminals (source and sinks) and the routing channels. The overlap of bounding boxes of nets N_1 and N_3 is highlighted in yellow.

(this is the decision of the designer) and on how the parallel computation, data synchronization, and conflict-avoidance are implemented. Often, nets are partitioned based on the independence of their bounding boxes. If the bounding boxes of two nets overlap (Figure 1), the possibility of conflicts between the nets is high and they should probably not be routed in parallel. Fine-grain parallel routers accelerate the routing of a single net, i.e. they provide sub-net-level parallelism. Their performance depends on how much parallelism can be extracted and on how and when the shared data is synchronized. To leverage acceleration opportunities offered by both parallelization strategies, some work propose combining them.

The ideal parallel router is not only *fast*, but also *scalable* and *deterministic*. A parallel program is fast if the load among parallel instances is well balanced and if the overhead caused by load partitioning is minimized. It is scalable if adding more processing resources leads to a proportionate increase in speedup; high scalability is an important advantage. It is deterministic if, for a given input, it always produces identical output; this repeatability is crucial in industrial environments, as it enables faster development and debugging cycles, as well as reliable testing. None of the existing parallel routers satisfies all these requirements entirely.

In this paper, key properties of the routers will be presented, along with discussion on whether they are deterministic and scalable, and what parallel programming models and synchronization methods are used. Then, the performance of all the routers will be compared, while mentioning the experimental methodology chosen by the corresponding authors.

III. COARSE-GRAIN PARALLEL ROUTERS

A. Partitioning Based on Lexicographic Order

Twenty years ago, Chan et al. [11] were the first to try parallelizing the delay-driven negotiation-based router. They targeted a DM with message passing model: a networked processor cluster. In this cluster, each processor was running a copy of the algorithm, had its private version of congestion information, and was responsible for routing a subset of nets. Asynchronous mode of operation was employed: as soon as

the congestion information would change for one processor, this processor would update its local data and notify others of the change. One processor would act as a master; it would receive and broadcast the congestion updates via UNIX sockets. This communication was slow and heavy-load (copies of changes were made for each destination). Moreover, processors were not synchronized after every congestion update, but less often (master would transmit averaged update values). All nets were first sorted in lexicographic order, thus keeping those nets that would potentially share resources adjacent and likely assigned to the same processor. Then, this ordered list of nets was partitioned in disjoint sets, statically assigned to processors. The disadvantages of their approach are the following: (1) the static net partitioning did not allow load balancing among processors for improved runtime, (2) the results were sensitive to the order of nets to be routed and (3) the results were non-deterministic as there was no guarantee regarding the order of arrival of the messages to a processor. The performance was tested by routing only two circuits. The average speedup, compared to the baseline sequential implementation of the original PathFinder algorithm, was quite good: $2.6\times$ for three processors and $3.2\times$ for four processors. A few years later, the same authors thoroughly examined the convergence of the PathFinder algorithm [12] and proposed an improved parallel algorithm, in which the nets were sorted according to increasing order of fanout and then distributed evenly among processors so that each processor had roughly the same number of sinks (balanced load). The speedup was similar to their initial work and the results were still non-deterministic.

B. Parallel Routing on Planar FPGAs

Cabral et al. [13] propose a parallel routing algorithm for FPGAs with planar routing architecture. In planar architecture, wires in the i -th routing track of one channel can connect only to the wires that are in the same i -th routing track. This restriction greatly simplifies the parallelization problem as it allows safe independent routing of nets. The speedup achieved is, as expected, almost linear. Yet, this interconnect architecture provides limited routability and is no longer used, making the approach inapplicable today.

C. Partitioning with Load Balancing

Gort et al. [14] propose concurrently running processes to accelerate the routing. These processes run on different cores or on different processors across a network (Figure 2). In their work, each process is an instance of a Versatile Place and Route (VPR) framework [15], responsible for routing its own set of signals and maintaining its own data structures (an RRG and associated congestion information). The synchronization among different VPR instances, i.e., the exchange of intermediate routing results and associated congestion, is achieved using MPI. The choice of MPI permits to keep unaltered the data-structures and functions in VPR.

After a VPR instance routes a signal, it issues a nonblocking send message (update) to all other VPR instances and con-

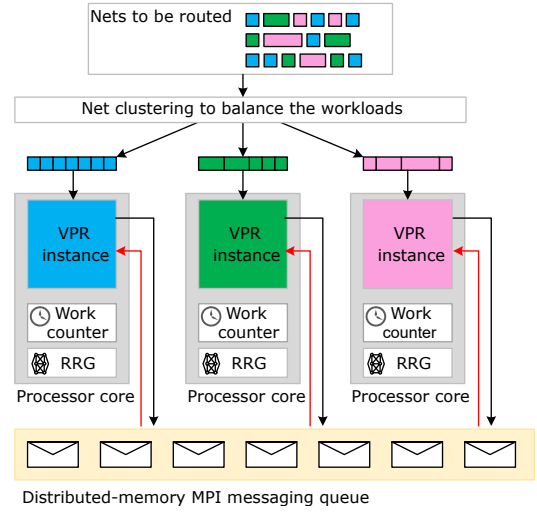


Fig. 2. Partitioning with load balancing. Example for $N = 3$ cores.

tinues without waiting to receive an update from other VPR instances. The update message contains the new route for the signal, along with some additional information. The messages are held in the MPI messaging queue. To receive an update, a VPR instance issues a *blocking* receive call at specific (fixed) points during routing. These points are identical from run-to-run—the key to achieving deterministic results. If the update is not available at the moment the VPR instance wants to receive it, since the receive call is blocking, the VPR instance will stall its execution and wait for the update to become available.

To minimize the stall time, the authors try to balance the amount of work among VPR instances, and have them issue a blocking receive calls only when it is likely that the update they want is available in the MPI messaging queue. To know when to issue a receive call, each VPR instance maintains work counters that estimate the amount of work performed by other VPR instances.

To balance the workload, the authors try to estimate (predict) signal route times using several different prediction metrics (fanout, distance between pins, number of RRG nodes visited during maze routing) and use these estimates to partition signals into as many sets as there are processor cores.

D. Recursive Partitioning

Shen et al. [16] explore parallelism by recursively partitioning the nets and routing some of these partitions in parallel. Recursive partitioning was not a novel idea, already Gort et al. [14] proposed it to improve algorithm convergence.

Firstly, routing resource nodes are assigned locations (placement coordinates of the logic blocks or routing tiles that these nodes belong to). Then, using the median cut-line the routing resource region is split into two subregions: upper and lower, or right and left, depending on the direction of the median cut-line. This creates three subsets of nets: the nets that cross the two subregions, the nets entirely in the first subregion, and the nets entirely in the second subregion. The first group of nets is normally smaller than the other two, and is routed

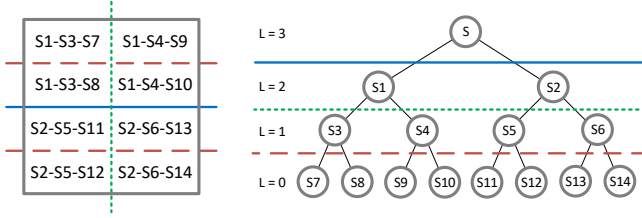


Fig. 3. Recursive partitioning for $N = 3$ cores (left) and the corresponding binary tree (right).

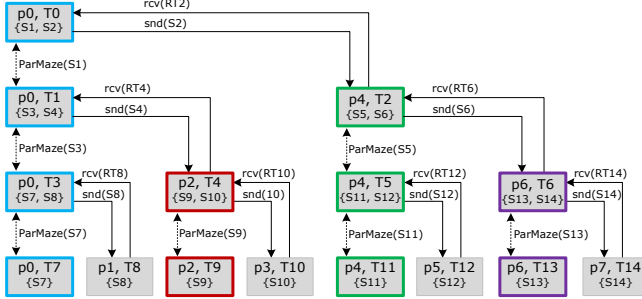


Fig. 4. Recursive partitioning for $N = 3$ cores: the programming flow.

sequentially. The remaining two sets are disjoint and can be routed in parallel. These steps can be recursively repeated, allowing for more parallelization.

Assuming three levels of partitioning, we can visualize the partitions as shown in Figure 3. As the first step, the blue solid line splits the routing region S (the complete chip) into two subregions: $S1$ and $S2$. Then, the dashed green line splits $S1$ into $S3$ and $S4$, and $S2$ into $S5$ and $S6$. Finally, the dashed red lines partitions $S3$ into $S7$ and $S8$, $S4$ into $S9$ and $S10$, $S5$ into $S11$ and $S12$, and $S6$ into $S13$ and $S14$. This partitioning scheme can also be viewed in a form of a 3-level binary tree.

To parallelize the routing, Shen et al. formulate the routing as tasks in a DM system: task $T0$ is responsible for routing all nets, task $T1$ for nets in subregion $S1$, task $T2$ for nets in subregion $S2$, task T_i for subregion S_i (Figure 4). These tasks are statically mapped to processes, thus enabling them to know whom to ask for partial results or to whom to send nets to be routed. MPI send/receive primitives are used for synchronization. Static allocation of tasks to processes and strict synchronization—routing of subregions cannot start before the nets crossing those two subregions are routed—are key to obtain deterministic results.

Figure 4 illustrates the parallel programming flow for the example in Figure 3. Routing starts by the master process $p = 0$ running task $T0$ in an L -level perfect binary tree (in Figure 4, $L = 3$). $T0$ partitions the nets in subregions $S1$ and $S2$. Then it first routes the nets that cross these two regions, before it assigns the right $(L - 1)$ -level subtree to process $p + 2^{(L-1)} = p + 4$ using an MPI message. Finally, it assigns the left $(L - 1)$ -level subtree to task $T1$ by recursive function call ($ParMaze()$ in Figure 4). Process $p + 4$ and task $T2$ return $RT2$, the routing tree for nets in $S2$. Task $T1$ returns $RT1$, the routing tree for nets in $S1$. These steps are repeated: $T1$

partitions $S1$ to $S3$ and $S4$, delegates the routing of nets in $S4$ to process $p + 2^{(L-1)} = p + 2$ and to task $T4$, and recursively calls the routing function for nets in $S3$ (task $T3$).

E. Hybrid Parallel FPGA Router

Hoo et al. proposed a two-phase hybrid parallel router, called ParaFRo [17], running on an SM system with threads. In the first routing phase, nets are partitioned, allocated to threads, and routed concurrently (Figure 5). Two scheduling policies were explored: static and greedy. In the static policy, each thread routes $(tid + k \times num_threads)$ -th net from the netlist, where tid is the thread identifier ranging from 0 to $num_threads - 1$. In greedy policy, to save time, threads get a new net from the netlist, as long as there is one. Threads are spawned using Intel Threading Building Block (TBB) library's *parallel_for* loop. Congestion updates are implemented using TBB's *spin_mutex* class, more time-efficient but less scalable than other mutexes.

To minimize the communication overhead, congestion information is updated only before moving on to the next routing iteration. If conflicts are detected, only congested nets are ripped and rerouted. If this phase converges, the routing is finished. If convergence is under question (the number of congested nodes is not decreasing through iterations), the algorithm switches to the second phase. In this phase, convergence is of higher priority than performance. Hence, the congested nets are split in as many groups as there are threads in such a way as to minimize the overlap of bounding boxes of nets that are in different partitions (this enhances convergence because it reduces the probability of reading outdated congestion state). Then, the nets are given to threads to route. To additionally help convergence, the entire route trees are periodically ripped.

The main drawback of this approach is that ParaFRo is not deterministic, because congestion costs seen by each thread depend on the time at which the thread discovers a routing resource node, and threads do not run at exactly the same speed. To make the algorithm deterministic, strict static scheduling and barriers should be put in place, and a copy of congestion state should be stored in each thread before routing. Obviously, this would affect the performance, but was not explored by the authors.

IV. FINE-GRAIN PARALLEL ROUTERS

Unlike coarse-grain parallelization, fine-grain parallelization does not modify the processing order of the nets. Since about 68% of routing time is consumed by calculating the cost of the neighbors of v_{min} and then inserting them into the priority queue [18], the maze expansion step of the PathFinder algorithm is the first candidate for fine-grain acceleration.

A. Parallel Maze Routing Using Galois Programming Model

Moctar et al. [19] were the first to use the open source Galois programming framework [20], [21] to accelerate the maze expansion step. They adapted the VPR 5.0 to be Galois-friendly by modifying the router [22] to be thread-safe and

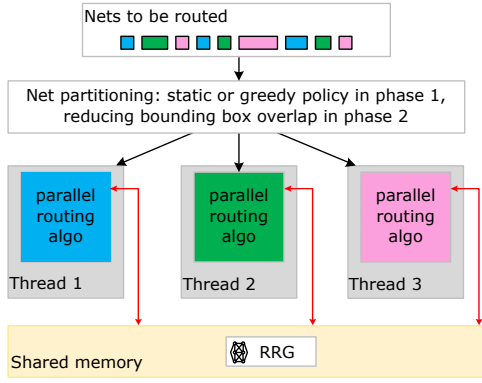


Fig. 5. Hybrid parallel FPGA router. Example for $num_threads = 3$.

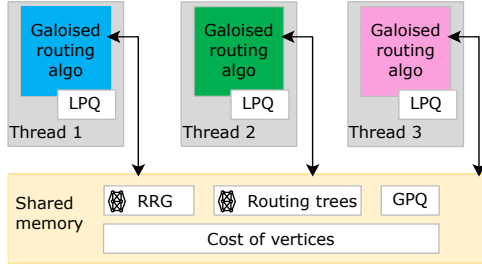


Fig. 6. Parallel maze routing using Galois. Example for three threads.

to use the *operator formulation*. Then, they let Galois extract fine-grain parallelism.

The algorithm was employed on a shared memory multi-processor system, with the number of cores varying from one to eight, and one thread per core (limitation imposed by Galois at the time). The multi-threaded parallelization strategy was based on Galois' *iteration coalescing*, which allows one thread to process multiple iterations at once. Each thread has its own local priority queue (LPQ), containing vertices to expand (Figure 6). If the LPQ is empty, threads access the global priority queue (GPQ), stored in the shared memory. Synchronization is implemented using locks. If a lock cannot be acquired, Galois runtime detects the conflict and rolls back one of the conflicting activities. This requires having always a copy of the data before modifications, i.e., an *undo log*. This approach is called *speculative parallelism*. Since Galois' implementation of priority queue (PQ) was a performance bottleneck, Moctar et al. replaced it with a more efficient PQ based on software transactional memory (STM).

At the time, Galois did not guarantee deterministic results. Hence, the algorithm produced different output depending on which thread would finish first. Additionally, this speculative parallelism approach scales well with the number of threads only when the possibility of contention is low, due to the high overhead of the rollback process.

B. Parallel Maze Routing Using Posix Threads

Gort et al. [18] proposed accelerating the cost calculation part of the maze expansion step using Posix threads (pthreads). If there are N available cores, they create one main thread and $N - 1$ helper threads (Figure 7). The main thread executes

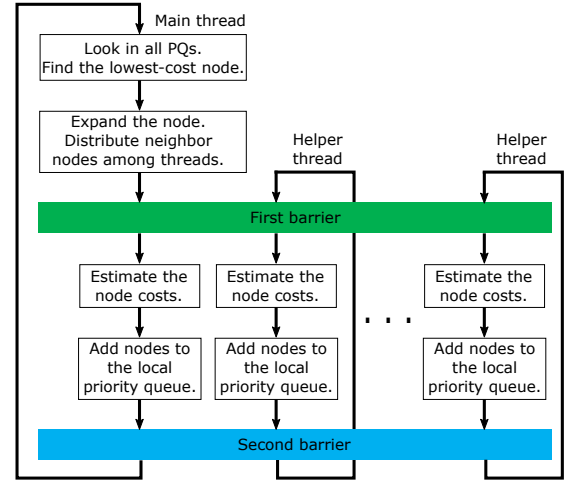


Fig. 7. Parallel maze routing using pthreads.

the serial portion of maze routing and $1/N$ -th of the parallel portion of the algorithm. The helper threads are created only once, and “busy-wait” during the serial portions of the algorithm. Each thread maintains its own priority queue. The fine-grain parallel routing starts by the main thread looking into the nodes on the top of all PQs and selecting the node with minimum cost (via a barrier). Then, the main thread expands this minimum-cost node and signals the helper threads that new nodes are ready to be processed (round-robin approach is used to assign nodes to threads). All threads find the cost of added nodes and place them in their PQs. Once the main thread is finished adding nodes to its own PQ, it waits in the second barrier for other threads to be finished, before repeating the described steps. To improve the speedup, the authors make threads spin in a loop while waiting to proceed through the barrier.

V. MIXED-GRAIN PARALLEL ROUTERS

A. Fanout-Aware Partitioning

Zhu et al. [23] propose using threads instead of processes, to reduce the communication overhead, and partitioning nets with respect to their fanout. They divide all nets in two categories: high-fanout nets (H-nets) and low-fanout nets (L-nets). Given that H-nets take a large percentage of total routing time [22], they further partition each of them into sets of lower fanout nets with smaller bounding boxes. These subnets can be routed in parallel, because they do not overlap. Therefore, the overall routing time of one H-net net can be reduced to the longest time to route any of its subnets. To accelerate routing of L-nets, which cannot be easily partitioned, they identify L-nets with non-overlapping bounding boxes and route those concurrently.

They parallelize the routing algorithm in VPR 5.0 [15] using pthreads [24]. In their solution, there is one main thread that creates all the helper threads (Figure 8). Two barriers are used to synchronize the threads. The first barrier informs the helper threads that the nets are ready to be routed. All threads wake up and route their nets. To avoid data competition and deadlock conditions (threads share the process memory space),

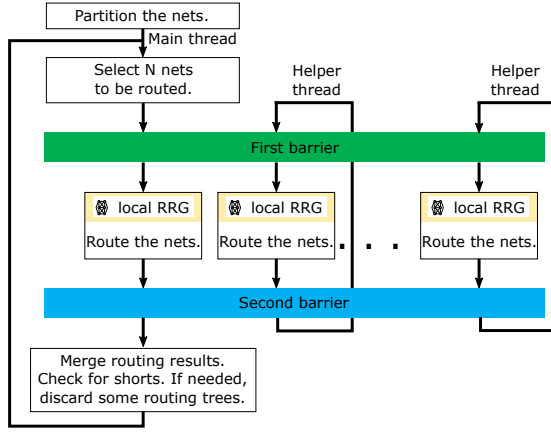


Fig. 8. Parallel mixed-grain routing using pthreads.

every helper thread has one copy of the RRG. In parallel, the main thread routes one partition of the task. The second barrier notifies the main thread when all threads have finished their work, allowing the main thread to merge partial results, update the congestion information, discard conflicting routing trees and reschedule the conflicting nets for routing, and put helper threads to sleep. Deterministic results are obtained.

VI. DIFFERENT APPROACHES TO PARALLEL ROUTERS

A. GPU-Accelerated Routing

Shen et al. were the first to use a GPU to accelerate FPGA routing [25]. Since the PathFinder algorithm is sequential in nature, and therefore not designed for GPU acceleration, they propose using the GPU-friendly single-source shortest path (SSSP) algorithm based on the Bellman-Ford algorithm instead. Their routing method guarantees deterministic results, albeit different than those from the original PathFinder algorithm. Like Moctar et al. [19], they resort to dynamic parallelism extraction and operator formulation.

To reduce the runtime, the problem size needs to be reduced as well. They argue that global routing in a coarsened (clustered, partitioned) routing graph is not the best approach, and instead resort to restricting the search space for the SSSP algorithm. To ensure the convergence and correctness of the routing algorithm even in a limited search space, they propose a *subgraph dynamic expansion* method. This method starts by limiting the subgraph size to the minimum-size rectangular bounding box, but then dynamically increases the subgraph size whenever a routed net within it touches its boundary. This way, the algorithm eventually manages to find a routing solution inside the subgraph.

To explore fine-grain parallelization, they combine three kinds of parallelism: static node parallelism (every node is statically assigned to a thread to be processed in parallel), dynamic node parallelism (only active nodes are assigned to threads), and dynamic edge parallelism (only active edges are assigned to threads).

On top of the fine-grain parallelization, they parallelize routing of multiple nets. To ensure deterministic results, they keep the original net ordering, and then create subsets of nets

that can be routed concurrently: the first element is the first net, the following elements are gradually added keeping the original order, until the routing subgraph of a net to be added has overlaps with one of the nets already in the subset. To unify single-net and multi-net routing, they introduce a virtual source node and connect it to the sources of all nets in a single subset. That way, multiple-net sets can be routed leveraging the fine-grain parallelization approach as well.

B. FPGA Routing as a Linear Programming Problem

Hoo et al. formulate a parallel FPGA routing algorithm as a linear programming problem and then apply Lagrangian relaxation to decompose the LP problem into independent subproblems, which can be solved in parallel [26]. The goal of the routing algorithm is to find a set of sub-trees in a weighted graph such that the total weight is minimized. The weights are set in such a way that these sub-trees could be determined in parallel using minimum Steiner tree algorithms. Since their algorithm updates weights only at the end of a routing iteration, the nets can be routed in parallel and in any order. Parallelization is done employing the *parallel_for* construct in Intel Threading Building Block (TBB) library. Due to this, their algorithm achieves the best speedup (close to the ideal linear speedup) only for very large circuits, when the overhead of task creation and context switching with Intel TBB are amortized. In addition, high fanout nets take more time to be routed using their minimum Steiner tree algorithm than using PathFinder in VPR [27]. Finally, although they achieve notable reduction of 22.4% on average in total wire length compared to VPR, minimum steiner trees cause critical path delays 7.5% higher than those of VPR, on average.

VII. COMPARISON OF PARALLEL FPGA ROUTERS

Table I summarizes the experimental platforms, benchmarks, and sequential baselines used to test the parallel routers. Very often, experiments were performed using MCNC benchmarks, which are composed of designs that are significantly smaller and simpler than current industrial-scale designs. Better evaluation has been done by researchers who chose Titan, VPR, or IWLS benchmarks.

Some authors chose to accelerate the routing-driven router, others the timing-driven router, and often their solution is applicable only to the target router type. Here we do not emphasize that difference, but try to make general observations based on the parallelism strategy and the achieved speedup.

Figure 9 gives an overview of the chosen parallel algorithm granularity, programming model, and synchronization—the features (1) that are key to obtaining deterministic results and (2) that affect the amount of scalability. Figure 10 gives the reported speedups for 2×, 3×, 4×, and 8× the parallelism.

Besides the speedup, one should estimate the effects that parallel router incurs on the number of routing tracks (channel width) and critical path delay. Only some researchers give this information. Gort et al. report up to 1% [14], [18], Shen et al. up to 2.73% [25], Hoo et al. up to 7.5% [26], and Shen et al. up to 10% [16], average increase in critical path delay. Moctar et

TABLE I
EXPERIMENTAL SETUPS USED TO EVALUATE THE PERFORMANCE OF PARALLEL FPGA ROUTERS.

	Chan et al. [11]	Cabral et al. [13]	Gort et al. [18]	Gort et al. [14]	Zhu et al. [23]	Moctar et al. [19]	Hoo et al. [26]	Shen et al. [16]	Hoo et al. [17]	Shen et al. [25]
HW platform	A network of Sun Microsystems UltraSparc 143MHz	A network of 200MHz IBM RS 6000 workstations each with 64Mb RAM	Two Linux systems, each with Core 2 Quad Q9550 at 2.83GHz and with 3GB memory	Two Linux systems, each with Core 2 Quad Q9550 at 2.83GHz and with 3GB memory	Intel Core i5-2400 quad-core processor with 32GB main memory	Server with 8 Intel Xeon E5540 quad-core processors at 2.53GHz, 40GB shared memory	Dual Intel Xeon CPU E5-2609 v2, 2.5GHz, 32GB RAM, Linux	Linux server, each node with two 6-core Intel Xeon E5-2430 at 2.2GHz and 32GB shared memory	Dual Intel(R) Xeon(R) CPU E5-2670 v3, 2.3GHz and 128GB of RAM	Linux server with a 6-core Intel Xeon E5-2620 CPU at 2.2GHz and 32GB SM and a Tesla K40c GPU
Benchmarks	alu4 s1A	MCNC	MCNC VPR 5.0	MCNC VPR 5.0	MCNC opencores	IWLS	MCNC	VPR 7.0	Titan	VPR 7.0
Seq. Baseline	Original PathFind.	VPR	VPR 5.0	VPR 5.0	VPR 5.0	VPR 5.0	VPR 7.0	VPR 7.0	VPR 7.0	VPR 7.0

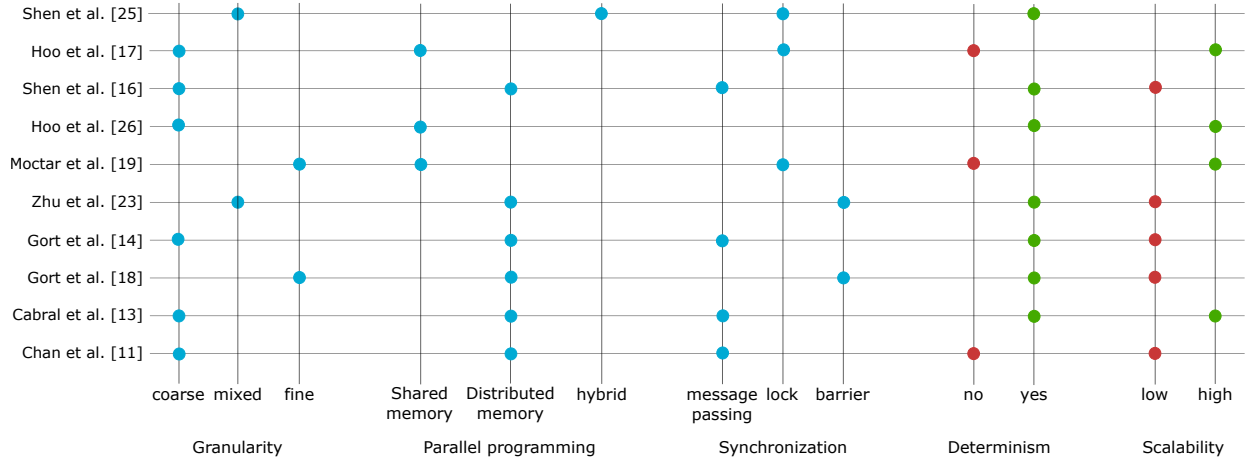


Fig. 9. Parallel FPGA routers compared in terms of granularity, programming model, synchronization, deterministic results, and scalability.

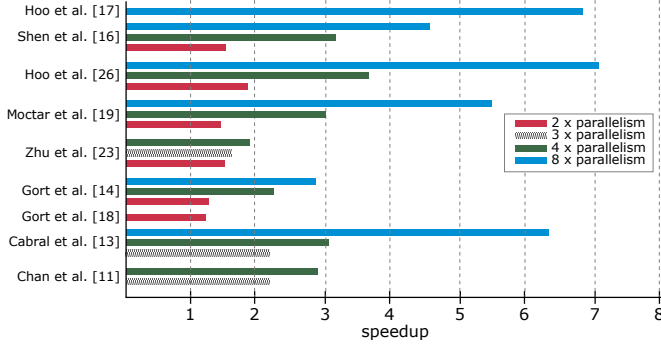


Fig. 10. Parallel FPGA routers compared in terms of speedup.

al. [19] and Hoo et al. [17] do not quantify the results, yet they achieve on average better critical path delay than the sequential baseline. Parallelization did not affect considerably the routing segment usage: only up to 1% variation is reported [14], [18], [23], [26]. For most applications, these numbers are an acceptable cost of the acceleration.

The choice of programming model and synchronization method (as characterized in Figure 9) is not sufficient to assess if results are deterministic: For example, Chan et al. [11] and Gort et al. [14] both use a distributed-memory

programming model and message passing, yet only the latter achieve determinism. Rather, what matters is that the congestion information seen by routing tasks is identical from run to run—i.e., synchronization imposes fixed net ordering.

Another interesting observation is that coarse-grain routers that divide nets based on their spatial location have low scalability (Figure 9). This happens because when the number of net sets increases (higher parallelism), it becomes harder to find the partitioning without overlaps. If two nets overlap, they might compete for the same routing resource, and thus the congestion information needs to be communicated more often. This leads to reduced speedup and, hence, lower scalability. Additional factor affecting scalability is the load balancing.

Comparing the speedup of the two fine-grain parallel routers (Figure 10), we see that Galois framework used by Moctar et al. [19] is an excellent platform for extracting hidden opportunities for acceleration, compared to the approach by Gort et al. [18]. Comparing the non-deterministic vs. deterministic routers confirms that determinism, due to its stricter synchronization requirements, reduces the speedup (Figure 10). One exception is the router by Hoo et al. [26], which achieves better speedup partly because the routing problem is differently formulated and approached. Another exception is the router by

Cabral et al. [13], but that one is not very relevant, seen that it is tailored to an obsolete FPGA routing architecture. Figure 10 is not showing the speedup of the router by Shen et al. [25], because they use CPU-GPU hybrid parallel programming model and an algorithm better suited for that platform. The speedup they achieve is impressive: $18.72\times$.

VIII. CONCLUSION

Parallel routers indeed offer important acceleration opportunities. Yet, they are challenging to implement and come at a cost: deterministic results require fixed net ordering, which imposes constraints on synchronization and reduces the ideal speedup. Existing work shows that good speedup is achieved by experimenting with (1) novel routing problem formulations, (2) parallel algorithms, (3) frameworks for dynamic extraction of parallelism, and (4) hybrid parallel programming models. However, a winning formula has not been found yet. A possibility, which has not been pursued often, could be to combine the strengths of coarse- and fine-grain parallel routers. Another possibility could be to use FPGAs themselves to assist in routing. Huang et al. [28] tried a radical version of this where the reconfigurable array is significantly modified from industrial designs so that nets route themselves through the array. In an era where heterogeneous massively-parallel platforms abound, less extreme and more practical options seem promising. An old problem such as accelerating FPGA routing appears still intellectually fecund and rich in opportunities.

REFERENCES

- [1] H. Esmailzadeh, E. Blem, R. S. Amant, K. Sankaralingam, and D. Burger, "Dark silicon and the end of multicore scaling," in *Proc. of the 38th Annual Intl. Symp. on Computer Architecture*, San Jose, California, Jun. 2011.
- [2] A. M. Caulfield, E. S. Chung, A. Putnam, H. Angepat, J. Fowers, M. Haselman, S. Heil, M. Humphrey, P. Kaur, J.-Y. Kim, D. Lo, T. Massengill, K. Ovtcharov, M. Papamichael, L. Woods, S. Lanka, D. Chiou, and D. Burger, "A cloud-scale acceleration architecture," in *Proc. of the 49th Annual Intl. Symp. on Microarchitecture*, Taipei, Taiwan, Oct. 2016.
- [3] K. E. Murray, S. Whitty, S. Liu, J. Luu, and V. Betz, "Timing-driven titan: Enabling large benchmarks and exploring the gap between academic and commercial CAD," *ACM Trans. on Reconfigurable Technology and Systems (TRETS)*, vol. 8, no. 2, pp. 10:1–10:18, Apr. 2015.
- [4] B. Catanzaro, K. Keutzer, and B.-Y. Su, "Parallelizing CAD: A timely research agenda for EDA," in *Proc. of the 45th Design Automation Conf.*, Anaheim, Calif., Jun. 2008, pp. 12–17.
- [5] J. Rose, "Parallel global routing for standard cells," *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems*, vol. 9, no. 10, pp. 1085–1095, Oct. 1990.
- [6] L. McMurchie and C. Ebeling, "PathFinder: A negotiation-based performance-driven router for FPGAs," in *Proc. of the 3th ACM/SIGDA Intl. Symp. on Field Programmable Gate Arrays*, Monterey, Calif., Feb. 1995, pp. 111–117.
- [7] C. J. Lee, "An algorithm for path connections and its applications," *IRE Trans. on Electronic Computers*, vol. EC-10, pp. 346–365, Aug. 1961.
- [8] Y. O. M. Moctar, G. G. F. Lemieux, and P. Brisk, "Fast and memory-efficient routing algorithms for field programmable gate arrays with sparse intracluster routing crossbars," *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems*, vol. 34, no. 12, pp. 1928–41, Dec. 2015.
- [9] V. Betz, J. Rose, and A. Marquardt, *Architecture and CAD for deep-submicron FPGAs*. Boston, Mass.: Kluwer Academic, 1999.
- [10] B. Barney, *Introduction to Parallel Computing*, Lawrence Livermore National Laboratory, 2016. [Online]. Available: https://computing.llnl.gov/tutorials/parallel_comp
- [11] P. K. Chan and M. D. F. Schlag, "Acceleration of an FPGA router," in *Proc. of the 5th IEEE Symp. on Field-Programmable Custom Computing Machines*, Napa Valley, Calif., Apr. 1997, pp. 175–81.
- [12] P. K. Chan, D. F. Schlag, C. Ebeling, and L. McMurchie, "Distributed-memory parallel routing for field-programmable gate arrays," *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems*, vol. 19, no. 8, pp. 850–862, Aug. 2000.
- [13] L. A. F. Cabral, J. S. Aude, and N. Maculan, "TDR: A distributed-memory parallel routing algorithm for FPGAs," in *Proc. of the 12th Intl. Conf. on Field-Programmable Logic and Appl.*, Montpellier, France, Sep. 2002, pp. 263–70.
- [14] M. Gort and J. H. Anderson, "Accelerating FPGA routing through parallelization and engineering enhancements," *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems*, vol. 31, no. 1, pp. 61–74, Jan. 2012.
- [15] J. Luu, I. Kuan, P. Jamieson, T. Campbell, A. Ye, W. M. Fang, K. Kent, and J. Rose, "VPR 5.0: FPGA CAD and architecture exploration tools with single-driver routing, heterogeneity and process scaling," *ACM Trans. on Reconfigurable Technology and Systems (TRETS)*, vol. 4, no. 4, pp. 32:1–32:22, Dec. 2011.
- [16] M. Shen and G. Luo, "Accelerate FPGA routing with parallel recursive partitioning," in *Proc. of the Intl. Conf. on Computer Aided Design*, Austin, Texas, Nov. 2015, pp. 118–25.
- [17] C. Hau Hoo, Y. Ha, and A. Kumar, "ParaFro: A hybrid parallel FPGA router using fine grained synchronisation and partitioning," in *Proc. of the 26th Intl. Conf. on Field-Programmable Logic and Appl.*, Lausanne, Aug. 2016, pp. 1–11.
- [18] M. Gort and J. H. Anderson, "Deterministic multi-core parallel routing for FPGAs," in *Proc. of the IEEE Intl. Conf. on Field Programmable Technology*, Beijing, Dec. 2010, pp. 78–86.
- [19] Y. O. M. Moctar and P. Brisk, "Parallel FPGA routing based on the operator formulation," in *Proc. of the 51st Design Automation Conf.*, San Francisco, Calif., Jun. 2014, pp. 1–6.
- [20] M. Mndez-Lojo, D. Nguyen, D. Prountzos, X. Sui, M. A. Hassaan, M. Kulkarni, M. Burtscher, and K. Pingali, "Structure-driven optimizations for amorphous data-parallel programs," in *Proc. of the 15th ACM SIGPLAN Symp. on Principles and Practice of Parallel Programming*, Bangalore, India, Jan. 2010, pp. 3–14.
- [21] K. Pingali, D. Nguyen, M. Kulkarni, M. Burtscher, M. A. Hassaan, R. Kaleem, T.-H. Lee, A. Lenharth, R. Manevich, M. Mndez-Lojo, D. Prountzos, and X. Sui, "The tao of parallelism in algorithms," in *Proc. of the ACM SIGPLAN'11 Conf. on Programming Language Design and Implementation*, San Jose, Jun. 2011, pp. 1–14.
- [22] J. S. Swartz, V. Betz, and J. Rose, "A fast routability-driven router for FPGAs," in *Proc. of the 6th ACM/SIGDA Intel. Symp. on Field Programmable Gate Arrays*, Monterey, Calif., Feb. 1998, pp. 140–49.
- [23] C. Zhu, J. Wang, and J. Lai, "A novel net-partition-based multithread FPGA routing method," in *Proc. of the 23rd Intl. Conf. on Field-Programmable Logic and Appl.*, Porto, Portugal, Sep. 2013, pp. 1–4.
- [24] C. Hughes and T. Hughes, *Professional Multicore Programming: Design and Implementation for C++ Developers*. Wiley, 2008.
- [25] M. Shen and G. Luo, "Corolla: GPU-accelerated FPGA routing based on subgraph dynamic expansion," in *Proc. of the 25th ACM/SIGDA Intl. Symp. on Field Programmable Gate Arrays*, Monterey, Calif., Feb. 2017.
- [26] C. Hau Hoo, A. Kumar, and Y. Ha, "ParaLaR: A parallel FPGA router based on Lagrangian relaxation," in *Proc. of the 25th Intl. Conf. on Field-Programmable Logic and Appl.*, London, Sep. 2015, pp. 1–6.
- [27] V. Betz and J. Rose, "VPR: A new packing, placement and routing tool for FPGA research," in *Proc. of the 7th Intl. Workshop on Field-Programmable Logic and Appl.*, London, Sep. 1997, pp. 1–10.
- [28] R. Huang, J. Wawrzyniek, and A. DeHon, "Stochastic, spatial routing for hypergraphs, trees, and meshes," in *Proc. of the 11th ACM/SIGDA Intl. Symp. on Field Programmable Gate Arrays*, Monterey, Calif., Feb. 2003, pp. 78–87.